

Bag of Tricks for Efficient Text Classification

a venue-paper view generated from `rif://joulin-2017-fasttext`

[Generated view — RIF reference rendering, profile (venue-paper, en).]

Third-party encoding of Joulin, Grave, Bojanowski, Mikolov, EACL 2017. Not an authored document.]

Abstract

We explore a simple and efficient baseline for text classification: a linear classifier over averaged bag-of-n-gram embeddings^{c2}. Our experiments show that this classifier, fastText, is often on par with deep learning classifiers in terms of accuracy on standard benchmarks^{c4}, while being many orders of magnitude faster: it trains on more than one billion words in less than ten minutes on a standard multicore CPU, and classifies half a million sentences among 312K classes in less than a minute^{c5}. The approach is released as an open-source library^{c7}.

1 Introduction

Text classification is a building block of many applications, from web search to information retrieval and document ranking. At the time of this work, neural models achieved strong accuracy on these tasks, but their training and inference cost limited their use on very large corpora; linear classifiers, although known to be strong baselines when properly tuned, were under-explored in those evaluations^{c1 E4}. We investigate how far a simple, efficient linear baseline can go, and show that it matches deep classifiers on several standard benchmarks at a fraction of the computational cost^{c4 c5}.

2 Related Work

Linear classifiers with well-designed features have a long history of strong results in text categorisation (McCallum and Nigam, 1998; Joachims, 1998)^{E4}. More recently, character-level convolutional networks (Zhang et al., 2015) and very deep convolutional networks (VDCNN; Conneau et al., 2017) reported state-of-the-art accuracy on large-scale classification benchmarks, at a substantial computational cost^{E1}. For large-output tagging, TagSpace (Weston et al., 2014) learns embeddings of words and hashtags^{E3}. Our model shares its core with the CBOW architecture of word2vec (Mikolov et al., 2013), replacing the middle word with a label^{c2}.

Dataset	fastText (bigr.)	VDCNN
AG	92.5	91.3
Sogou	96.8	96.8
DBpedia	98.6	98.7
Yelp P.	95.7	95.7
Yelp F.	63.9	64.7
Yahoo A.	72.3	73.4
Amazon F.	60.2	63.0
Amazon P.	94.6	95.7

Table 1: Accuracy on the eight benchmarks (values as reported^{E1}). Renderer-generated table from evidence E1 (figure G2, kind: generable).

3 Model

We represent a text as a bag of n-gram features; each feature has an embedding, the embeddings are averaged into a single text representation, and a linear classifier maps it to the labels^{c2}. Two components keep the model efficient at scale^{c3}: (i) a hierarchical softmax over the labels, based on Huffman coding, which reduces the complexity of computing the label distribution from $O(kh)$ to $O(h \log k)$, where k is the number of classes and h the dimension of the representation; and (ii) the hashing trick, which bounds the memory of the n-gram feature space. Word order is captured only locally, through the n-gram features themselves^{W2}.

4 Experiments

Sentiment and topic classification. We evaluate on eight standard datasets (AG, Sogou, DBpedia, Yelp P., Yelp F., Yahoo A., Amazon F., Amazon P.), following the protocol of Zhang et al. (2015)^{E1}. With bigram features, fastText reaches 92.5 (AG), 96.8 (Sogou), 98.6 (DBpedia), 95.7 (Yelp P.), 63.9 (Yelp F.), 72.3 (Yahoo A.), 60.2 (Amazon F.) and 94.6 (Amazon P.), on par with the best reported deep baselines on most datasets^{c4}. The deepest CNNs retain an edge on the large fine-grained datasets: VDCNN reports 64.7 on Yelp F. and 63.0 on Amazon F.^{c4 W1} All values are as reported in the encoded source^{E1}.

Speed. Training requires seconds per epoch where character-based CNNs require hours^{E2}; overall, fastText trains on more than one billion words in less than ten minutes on a standard multicore CPU, and evaluates half a million sentences among 312K classes in less than a minute^{C5}.

Tag prediction. On the YFCC100M tag-prediction task, fastText with bigrams ($h=200$) reaches 46.1 precision@1 against 35.6 for TagSpace ($h=200$), training in 13m38s^{C6 E3}.

5 Discussion

The comparison this work insists on is not accuracy alone but the accuracy/cost ratio: near-parity at orders-of-magnitude lower cost^{R1}. Three watchpoints qualify the results^{W1 W2 W3}: parity is not universal, as the gap towards the deepest CNNs grows on large fine-grained datasets^{W1}; competitive accuracy requires bigram features, the unigram model alone being substantially weaker^{W2}; and the effect of hashing collisions on the n-gram space is not quantified^{W3}. No claim of across-the-board state of the art is made^{C4}.

6 Conclusion

We presented a simple and efficient baseline for text classification, on par with deep classifiers on standard benchmarks and many orders of magnitude faster^{C4 C5}. Baselines of this kind should be systematically reported before adopting deeper architectures^{R1}. The implementation is available as the open-source fastText library^{C7 E5}.

References

- [1] X. Zhang, J. Zhao, Y. LeCun. 2015. Character-level Convolutional Networks for Text Classification. *NeurIPS*.
- [2] A. Conneau, H. Schwenk, L. Barrault, Y. LeCun. 2017. Very Deep Convolutional Networks for Text Classification. *EACL*.
- [3] J. Weston, S. Chopra, K. Adams. 2014. #TagSpace: Semantic Embeddings from Hashtags. *EMNLP*.
- [4] T. Mikolov, K. Chen, G. Corrado, J. Dean. 2013. Efficient Estimation of Word Representations in Vector Space. *ICLR Workshop*.
- [5] T. Joachims. 1998. Text Categorization with Support Vector Machines. *ECML*.
- [6] A. McCallum, K. Nigam. 1998. A Comparison of Event Models for Naive Bayes Text Classification. *AAAI Workshop*.

RIF claim graph (superscripts). Watchpoints W1–W3 are present as required by the watchpoint-invariance rule. Figure G1 (model architecture, author-original) is omitted here because verbatim reuse requires permission; the renderer never redraws original figures. Style emulated from the `style.yaml` corpus.